Programming Bootcamp 1

Week 8



RMIT UNIVERSITY

Version updated by December 2024

AGENDA

- Introduction
- Revise Key Concepts
- Q&A
- Lab Time
- CES - Course Survey
- Quiz

RMIT UNIVERSITY

Introduction

RMIT UNIVERSITY

Object-oriented concepts

Encapsulation (Week 8)

Inheritance (Week 10)

Polymorphism (Week 10)

Abstraction (Week 10)

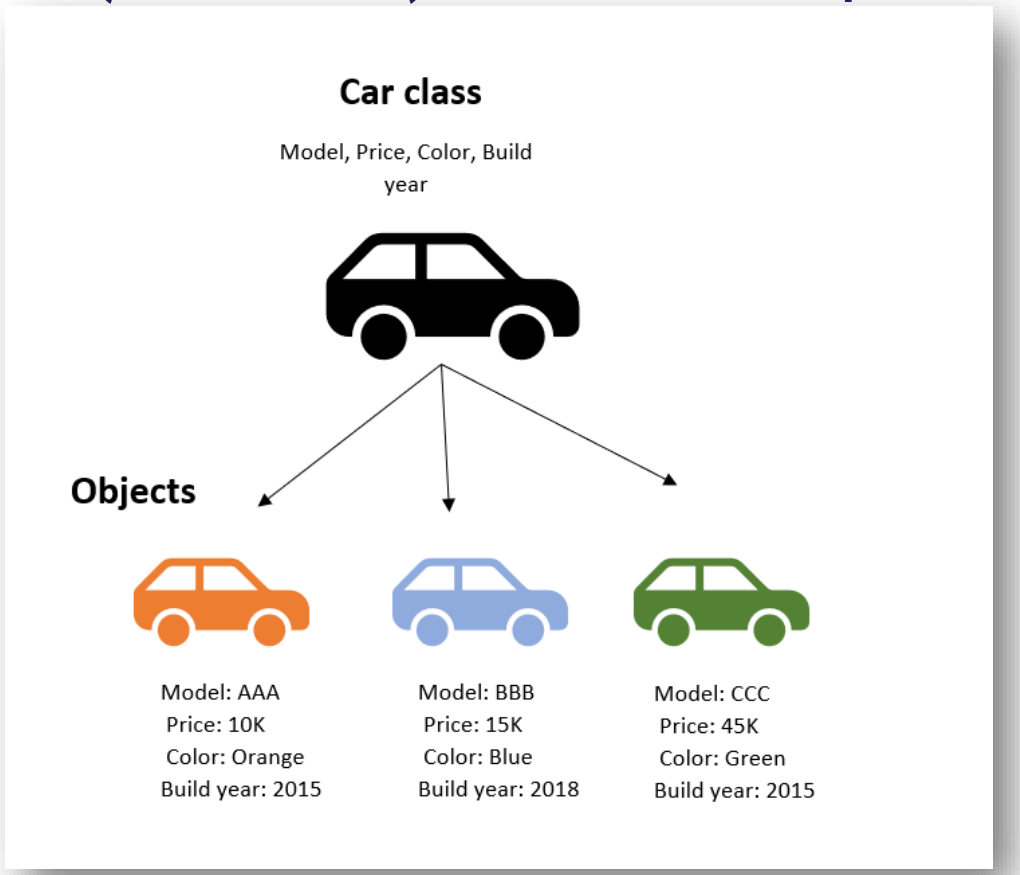


Object

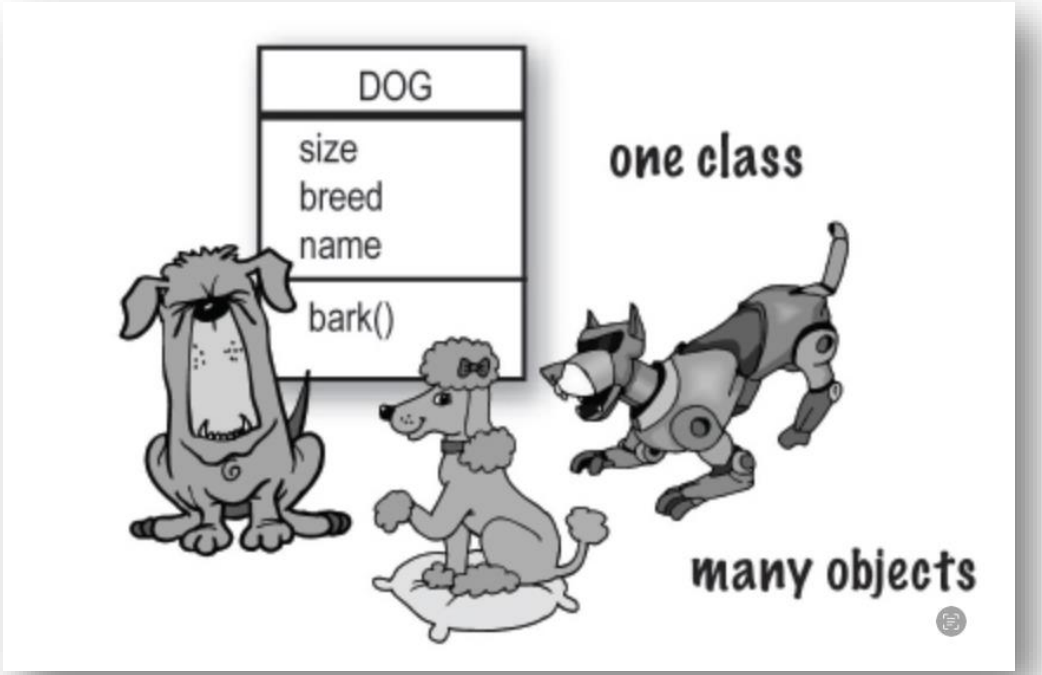
RMIT UNIVERSITY

Object

An **object** is a grouping of **data** (variables) and **operations** (methods) that can be performed on that data.



Car class vs. Car objects



Dog class vs. Dog objects

Abstraction

High-level representation that **hides** internal low-level details.

PARTICIPATION ACTIVITY | 10.1.4: Objects strongly support abstraction / information hiding.

Start ☐ 2x speed



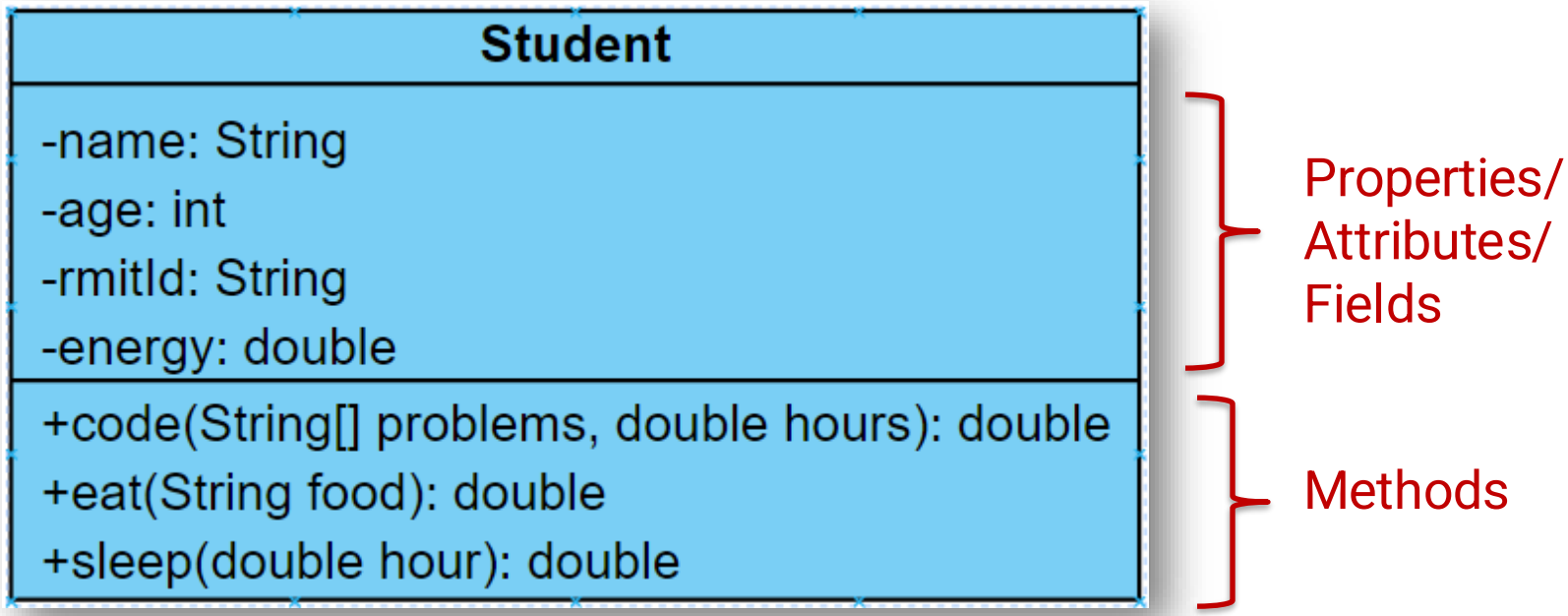


Don't do this

Class = Object definition

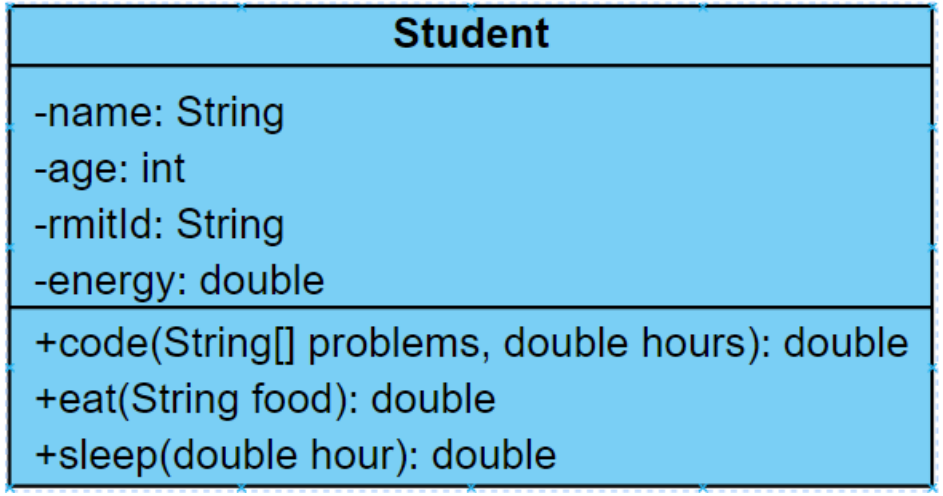
Class represents the **object type**

Class declares **properties** and **methods** of an object



Constructor

To instantiate/create an object

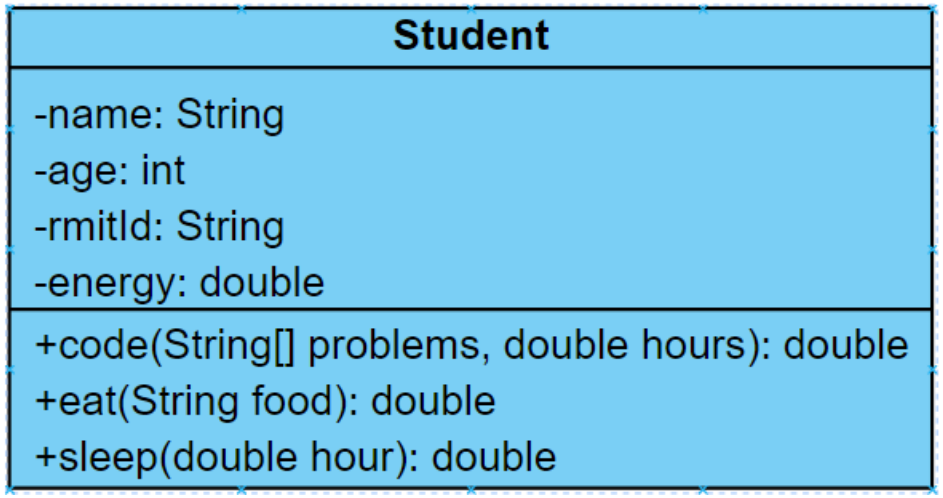



```
public class Student {
   String name;
   public Student(){
      name = “Default name”;
   }
}

Student student = new Student();
```

Constructor overloading

Multiple constructor definitions




```
public class Student {
   String name;
   public Student(){
      name = "Default name";
   }
   public Student(String inputName){
      name = inputName;
   }
}

Student student1 = new Student();
Student student2 = new Student("Phong");
```

Constructor overloading – Default constructor

Figure 10.9.1: Error - The programmer defined a constructor, so the compiler does not automatically define a default no-arg constructor.

```
public class Restaurant {
   public Restaurant(String initName, int initRating) {
      ...
   }

   // No other constructors
}

public class RestaurantFavorites {
   public static void main(String[] args) {
      Restaurant foodPlace = new Restaurant();
      ...
   }
}
```

```
RestaurantFavorites.java:3: cannot find symbol
symbol  : constructor Restaurant()
location: class Restaurant
      Restaurant foodPlace = new Restaurant();
                             ^
```

Member access operator "."

To access properties or methods on an object

PARTICIPATION ACTIVITY | 10.2.3: Using the Restaurant class.

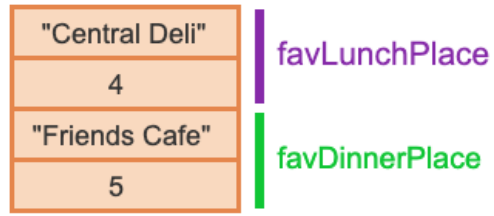
Start | 2x speed

```
public class RestaurantFavorites {
   public static void main(String[] args) {
      Restaurant favLunchPlace = new Restaurant();
      Restaurant favDinnerPlace = new Restaurant();

      favLunchPlace.setName("Central Deli");
      favLunchPlace.setRating(4);

      favDinnerPlace.setName("Friends Cafe");
      favDinnerPlace.setRating(5);

      System.out.println("My favorite restaurants: ");
      favLunchPlace.print();
      favDinnerPlace.print();
   }
}
```



My favorite restaurants:
Central Deli -- 4
Friends Cafe -- 5

Exercise



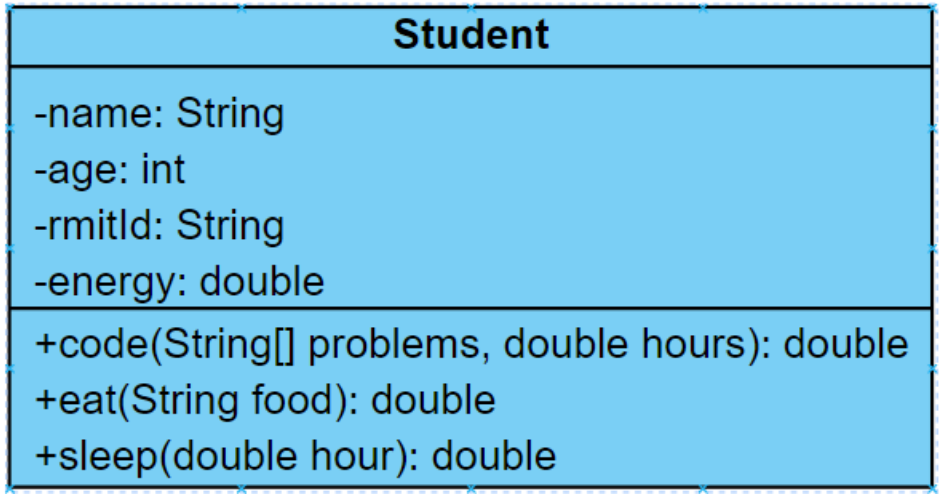
https://www.w3schools.com/java/exercise.asp?x=xrcise_constructors1

Access modifier

Apply to the **properties**, **methods**, and **constructors**

Public (+): accessible for class user

Private (-): accessible only for member methods not class user



What is “this”?

Refer to the current object in a method or constructor

Figure 10.11.1: Using 'this' to refer to an object's members.

ShapeSquare.java:

```
public class ShapeSquare {
   // Private fields
   private double sideLength;

   // Public methods
   public void setSideLength(double sideLength) {
      this.sideLength = sideLength;
      // Field member    Parameter
   }

   public double getArea() {
      return sideLength * sideLength; // Both refer to field
   }
}
```

ShapeTest.java:

```
public class ShapeTest {
   public static void main(String[] args) {
      ShapeSquare square1 = new ShapeSquare();

      square1.setSideLength(1.2);
      System.out.println("Square's area: " + square1.getArea());
   }
}
```

```
Square's area: 1.44
```

Exercise



https://www.w3schools.com/java/exercise.asp?x=xrcise_modifiers1

Accessor & Mutator

RMIT UNIVERSITY



Accessor: Get the **value** of an object's **property**

```
public int getAge() {
    return age;
}
```



Accessor: Get the **value** of an object's **property**

```
public int getAge() {
    return age;
}
```

Mutator: Modify the **value** of an object's **property**

```
public void setAge(int age) {
    this.age = age;
}
```

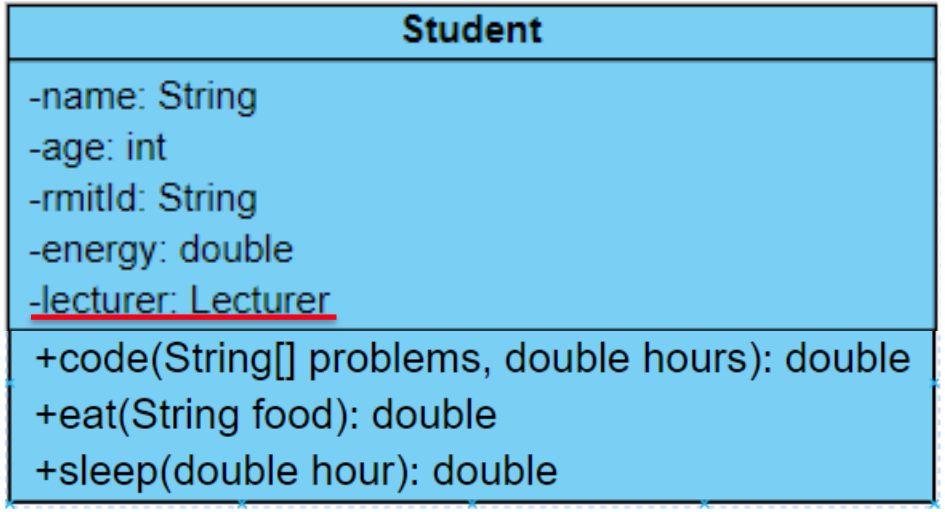
Both Accessor & Mutator are methods

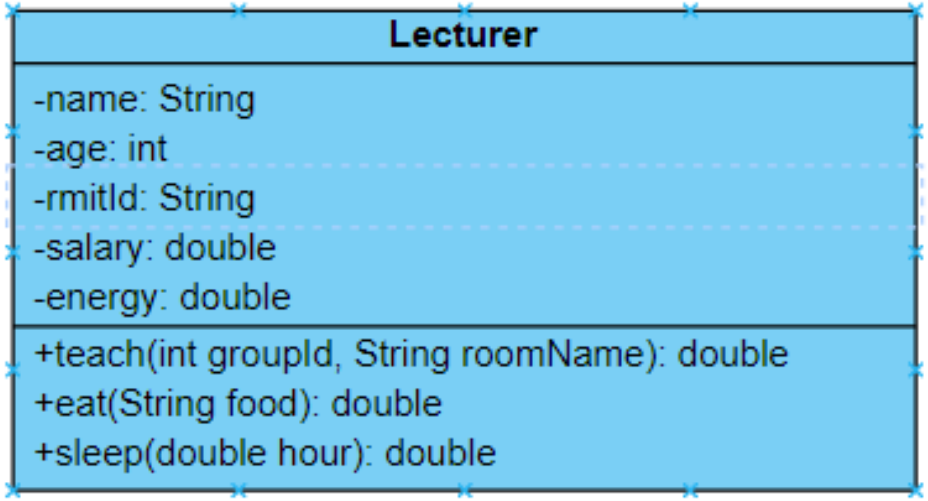
Getter and setter methods can be generated quickly by IntelliJ

Object within an Object

RMIT UNIVERSITY

An object can have another object as its property





Objects and References

RMIT UNIVERSITY

A **reference** is a variable type that refers to an **object**

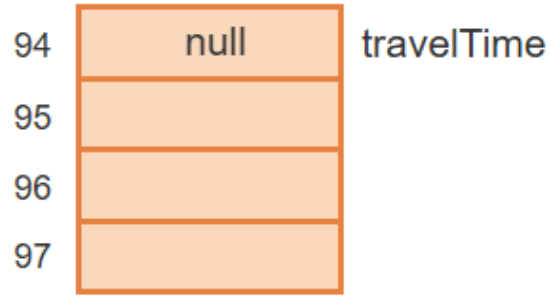


```
// Reference variable does not refer
// to any object upon definition
TimeHrMin travelTime;

// New allocates memory for object, returns
// reference to object
travelTime = new TimeHrMin();

travelTime.setHour(2);
travelTime.setMinute(40);

travelTime.printTime();
```



PA 10.10.1

```
// Reference variable does not refer
// to any object upon definition
TimeHrMin travelTime;

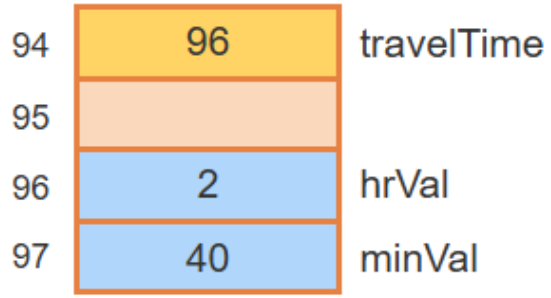
// New allocates memory for object, returns
// reference to object
travelTime = new TimeHrMin();

travelTime.setHour(2);
travelTime.setMinute(40);

travelTime.printTime();
```

Memory for TimeHrMin object instance

Memory for reference variable



Null Value

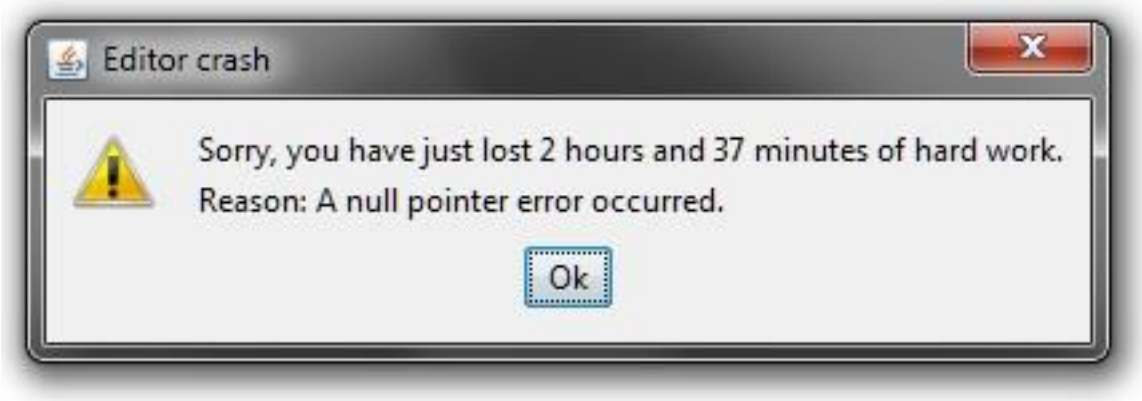
RMIT UNIVERSITY

Null means *absence* of a value or a *non-existent* object

```
Student stu1;
stu1.eat("Hamburger");
```

NullPointerException

```
Student stu2 = new Student();
stu2.eat("Hotdog");
```



Exercise



https://www.w3schools.com/java/exercise.asp?x=xrcise_class_methods1

Static Field and Method

RMIT UNIVERSITY

Static properties are **shared** among **objects**

PARTICIPATION ACTIVITY | 10.20.1: Static field used to create object ID numbers.

Start | 2x speed

Store.java

```
public class Store {
   // Declare and initialize public static field
   public static int nextId = 101;

   // Define private fields
   private String name;
   private String type;
   private int id;

   public Store(String storeName, String storeType) {
      name = storeName;
      type = storeType;
      id = nextId;

      ++nextId;   // Increment each time a Store object is created
   }

   ...
}
```

NewStores.java

```
public class NewStores {
   public static void main(String[] args) {
      Store store1 = new Store("Macy's", "Department");
      Store store2 = new Store("Albertsons", "Grocery");
      Store store3 = new Store("Ace", "Hardware");

      System.out.println("Store 1's ID: " + store1.getId());
      System.out.println("Store 2's ID: " + store2.getId());
      System.out.println("Store 3's ID: " + store3.getId());
      System.out.println("Next ID: " + Store.nextId);
   }
}
```



Static method does not require object declaration to call



Figure 10.20.1: Static member method used to access a private static field.

Store.java

```
public class Store {
   // Declare and initialize private static field
   private static int nextId = 101;

   // Define private fields
   private String name;
   private String type;
   private int id;

   public Store(String storeName, String storeType) {
      name = storeName;
      type = storeType;
      id = nextId;

      ++nextId;    // Increment each time a Store object is created
   }

   public int getId() {
      return id;
   }

   public static int getNextId() {
      return nextId;
   }
}
```

ArrayList

RMIT UNIVERSITY

Provide convenient array operations without fixed size



Table 10.15.1: Common ArrayList methods.

add()	`add(element)`
	Create space for and add the element at the end of the list.
		`// List originally empty`
		`valsList.add(31); // List now: 31`
		`valsList.add(41); // List now: 31 41`

get()	`get(index)`
	Returns the element at the specified list location known as the ***index***. Indices start at 0.
		`// List originally: 31 41 59. Assume x is an int.`
		`x = valsList.get(0);  // Assigns 31 to x`
		`x = valsList.get(1);  // Assigns 41`
		`x = valsList.get(2);  // Assigns 59`
		`x = valsList.get(3);  // Error: No such element`

set()	`set(index, element)` Replaces the element at the specified position in this list with the specified element.	`// List originally: 31 41 59` `valsList.set(1, 119);  // List now 31 119 59`
size()	`size()` Returns the number of list elements.	`// List originally: 31 41 59. Assume x is an int.` `x = valsList.size();  // Assigns x with 3`

Exercise



https://www.w3schools.com/java/exercise.asp?x=xrcise_arraylist1

Q & A

RMIT UNIVERSITY

Lab Time

RMIT UNIVERSITY

Course Survey

RMIT UNIVERSITY

ation: Trusted

CES – Course Experience Survey

- Please do it as soon as possible!
- It is **very important** for you, me and RMIT
- NOTE: RMIT interprets 1-3 (means no, bad) and 4-5 (means yes, good)
- Please mention things you like so I will try to make it happen more often
- Also, mention things you think it could be improved or changed
- Use link: https://surveys.rmit.edu.au



Quiz

RMIT UNIVERSITY

Have fun coding &
See you next week

RMIT UNIVERSITY